

Laboratory 3: Interrupts and Timers

NUCLEO-F411RE · STM32CubeIDE with the HAL library

Objectives

- Explain when to service a peripheral by polling and when to service it by interrupt.
- Configure the EXTI controller so that a push-button generates an interrupt.
- Configure a general-purpose timer to produce precise periodic events and output signals.
- Receive UART data by interrupt instead of by blocking calls.

1 Interrupts

1.1 Polling vs. interrupts

A CPU executes one instruction at a time, so it needs some way of knowing when a peripheral requires attention. There are two ways to arrange this.

Polling. The CPU repeatedly reads the status flag of the device and services it when the flag is set. This is simple to write, but the CPU spends most of its time asking “are you ready yet?”, it cannot sleep, and the response time depends on how quickly the polling loop comes back around.

Interrupts. The device raises an interrupt request (IRQ). The CPU finishes the current instruction, saves its context, jumps to an interrupt service routine (ISR) that services the device, then resumes exactly where it left off. Between events the CPU is free to do other work or to sleep.

Analogy: polling is picking up a phone that has no ringer every few seconds to check whether anyone is on the line. An interrupt is the ringer.

Typical uses: a button press that toggles an LED, a character arriving at a serial port, a low-battery detector that triggers a safe shutdown.

1.2 Nested Vectored Interrupt Controller (NVIC)

The NVIC is the block inside the Cortex-M4 core that decides which interrupt runs and when. Every interrupt, including the core exceptions, passes through it.

- Up to 62 maskable channels on the STM32F411, each enabled and disabled individually.
- 16 priority levels (4 priority bits). A higher-priority request pre-empts an ISR that is already running — this is what “nested” means.
- Low-latency entry and exit: the core stacks registers automatically and chains back-to-back interrupts without returning to the main program in between.

Programming details are in the Cortex-M4 programming manual PM0214; the interrupt vector table for this device is in RM0383.

1.3 External Interrupt/Event Controller (EXTI)

The EXTI turns a change on a pin into an interrupt request. It contains up to 23 edge detectors, and each line is configured independently:

- interrupt (runs an ISR) or event (wakes the core / triggers another peripheral, no ISR);
- trigger on the rising edge, the falling edge, or both;
- masked or unmasked; a pending register records requests that have not been served yet.

Two things to watch. First, EXTI lines are shared by pin number: PA13, PB13 and PC13 all map to EXTI13, and only one of them can use it at a time. Lines 10–15 also share a single vector, so EXTI15_10_IRQHandler() must work out which pin caused the request. Second, the pending bit must be cleared or the ISR fires forever — HAL_GPIO_EXTI_IRQHandler() does this for you before it calls your callback.

Rules for writing an ISR

- Keep it short. Set a flag, copy one byte, then return; do the real work in main().
- Any variable shared between an ISR and main() must be declared volatile.
- Never block inside an ISR — no HAL_Delay(), no printf(), no waiting loops.

2 Timers

2.1 Basic structure

A timer is a counter driven by a periodic clock. Starting from zero, the counter increments on every clock transition of one chosen direction (low-to-high or high-to-low, not both). If the clock frequency f is known, a count of k means that a time of k/f has elapsed — which is where the name comes from.

Practical timers add a few blocks around that counter, as shown below:

- a multiplexer that selects one of several clock sources;
- a prescaler (PSC) that divides the input clock before it reaches the counter;
- an n -bit counter (CNT), the counting element itself;
- a compare/reload register (ARR) that sets the highest value the counter may reach;
- a comparator that signals the match and resets the counter — this match is the update event that raises the interrupt.

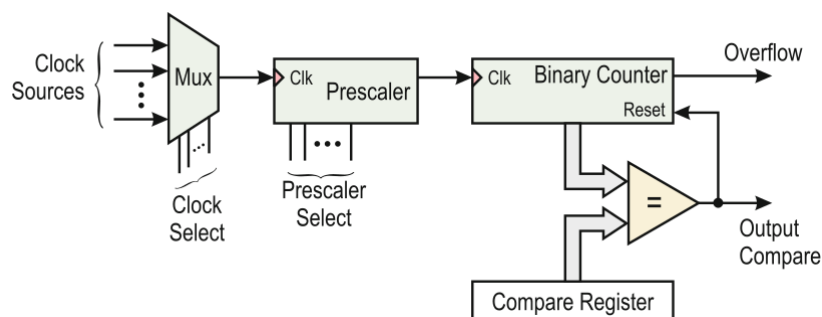


Fig. 1 Components of a base timer.

2.2 Choosing PSC and ARR

The update event (one full cycle of the counter) has period:

$$T = (PSC + 1) \times (ARR + 1) / f_{TIM} \quad f_{event} = 1 / T$$

Work in two steps. Pick PSC first so that one count is a convenient unit of time, then pick ARR to reach the period you want. On this board the timers run at 100 MHz, so $PSC = 9999$ gives $100 \text{ MHz} / 10000 = 10 \text{ kHz}$, i.e. one count = 100 μs . From that single time base:

- 500 ms $\rightarrow ARR = 5000 - 1 = 4999$
- 490.5 ms $\rightarrow ARR = 4905 - 1 = 4904$
- 999.9 ms $\rightarrow ARR = 9999 - 1 = 9998$

PSC and ARR are 16-bit (0–65535) on TIM3, TIM4 and TIM9–TIM11; TIM2 and TIM5 have a 32-bit counter. If your numbers do not fit, raise PSC.

2.3 Timer types on STM32

- **General-purpose** — output compare, PWM, one-pulse mode, input capture, encoder and Hall-sensor interfaces. On the STM32F411 these are TIM2–TIM5 and TIM9–TIM11; use them for this lab.
- **Advanced** — everything above plus complementary outputs with dead-time insertion and an emergency shutdown input, for motor control and power conversion (TIM1 here).
- **Basic** — no inputs or outputs; a plain time base, also used to trigger the DAC. Present on larger STM32F4 parts, not on the F411.
- **Low-power** — keeps running in low-power modes and can wake the core.
- **High-resolution** — very fine timing resolution for lighting and power supplies.

3 Lab exercises

Create a separate STM32CubeIDE project for each exercise. All four use the pins in the table below; set the system clock to 100 MHz in the Clock Configuration tab of every project.

Signal	Pin (board label)	Notes
USER button B1	PC13 → EXTI13	On the board already. Pressed = 0 V, released = 3.3 V, so trigger on the falling edge. The vector is EXTI15_10_IRQn, shared by lines 10–15.
LED 1 (external)	PA9 (D8)	Any colour. Pin → 330 Ω → LED anode, cathode → GND. LED lights when the pin is high.
LED 2 (external)	PA8 (D7)	Same wiring.
LED 3 (external)	PB10 (D6)	Same wiring.
Timer output	PA6 (D12) = TIM3_CH1	Used in Exercise 2. Probe here with the oscilloscope; connect the probe ground to a GND pin on the board.
Serial port	USART2: PA2 = TX, PA3 = RX	Already wired to the ST-LINK virtual COM port, so the terminal runs over the same USB cable. 115200 baud, 8 data bits, no parity, 1 stop bit — same settings as Lab 2.
Timer clock	100 MHz	After setting the system clock to 100 MHz (HSI + PLL). TIM1, TIM9–TIM11 and TIM2–TIM5 all receive 100 MHz with the default CubeMX prescalers.

Wiring the three LEDs

Each LED needs its own series resistor: MCU pin → 330 Ω (220–470 Ω is fine) → LED anode (long leg), LED cathode (short leg, flat side) → GND on the board.

The colours do not matter; label them LED 1, LED 2 and LED 3 in your code and in your report. Configure all three pins as GPIO_Output, push-pull, no pull, low speed.

Never connect an LED to a pin without a resistor — the pin is limited to about 20 mA and will be damaged.

Exercise 1: Push-button changes the blink rate (EXTI)

Blink LED 1. Each press of the USER button (B1) moves the blink period to the next value in this cycle:

0.2 s → 1 s → 5 s → 0.2 s → ...

Requirements

- The button must be handled by EXTI. Polling the pin in the main loop is not accepted.
- Configure PC13 as GPIO_EXTI13 with a falling-edge trigger and the internal pull-up enabled, then tick EXTI line[15:10] interrupts in the NVIC tab.
- In `HAL_GPIO_EXTI_Callback()`, only advance an index into a table of periods; do the blinking in `main()`. Declare the shared index volatile.
- Handle contact bounce — ignore any edge that arrives less than about 50 ms after the previous one (compare `HAL_GetTick()` values).

Check: the period changes on the press, not on the release, and one press never skips two steps.

Exercise 2: 100 µs clock signal on an output pin

Generate a continuous square wave with a period of 100 µs (10 kHz) on PA6 (TIM3_CH1, pin D12) and measure it with an oscilloscope.

Requirements

- The waveform must be produced by timer hardware (output compare in toggle mode, or PWM at 50 % duty). Toggling a GPIO inside a delay loop is not accepted.
- Suggested settings with a 100 MHz timer clock: PWM mode with `PSC = 0`, `ARR = 9999`, `CCR = 5000`, started by `HAL_TIM_PWM_Start()`.
- Careful with toggle mode: the output flips once per update event, so the output period is twice the update period. For a 100 µs output you need a 50 µs update, i.e. `ARR = 4999`.

Check: capture the waveform and include the measured period and frequency in your report. Comment on any difference from 100 µs and explain where it comes from.

Exercise 3: Two LEDs and a counter report (timer interrupt)

Using timer interrupts only, run three periodic activities at the same time:

1. toggle LED 1 every 500 ms;
2. toggle LED 2 every 490.5 ms;
3. every 999.9 ms, send both blink counts to the serial terminal over USART2, for example: LED1=12 LED2=13.

Requirements

- All three periods come from timer update interrupts (`HAL_TIM_PeriodElapsedCallback`), enabled with `HAL_TIM_Base_Start_IT()`. No `HAL_Delay()` anywhere in the loop.
- Use either three timers, or one 100 µs time base with three software counters — state which you chose and why.
- Keep the callback short: increment counters and toggle pins there, and send the text from `main()` when a flag is set.

Check: because the two periods differ by 9.5 ms, the LEDs start together and drift visibly apart. Explain in your report how long it takes them to end up in anti-phase.

Exercise 4: UART echo using the receive interrupt

Repeat the echo program from Lab 2 — every character received on USART2 is sent straight back — but receive by interrupt instead of by a blocking call.

Requirements

- Enable the USART2 global interrupt in the NVIC tab and start reception with `HAL_UART_Receive_IT(&huart2, &rxByte, 1)`.
- Echo the byte in `HAL_UART_RxCpltCallback()` and re-arm reception at the end of the callback, otherwise only the first character arrives.
- While waiting for characters, blink LED 3 in `main()` to show that the CPU is not blocked.

Check: type continuously in the terminal; every character comes back, and LED 3 keeps blinking at a steady rate throughout.

4 What to hand in

- The four source projects (or at least `main.c` and the `.ioc` file of each).
- A photo or sketch of your LED wiring.
- For each exercise: the PSC/ARR or EXTI settings you used and the calculation behind them.
- The oscilloscope capture from Exercise 2 and the terminal output from Exercises 3 and 4.
- Short answers to the “Check” item at the end of each exercise.

Background material adapted from M. Jiménez, R. Palomera and I. Couvertier, Introduction to Embedded Systems Using Microcontrollers and the MSP430, and from the STM32F411 reference manual (RM0383) and the NUCLEO-64 user manual (UM1724).